

Concur: 通过基于拥塞的并发控制实现高吞吐 Agent 批量推理

Qiaoling Chen¹ Zhisheng Ye² Tian Tang³ Peng Sun³ Boyu Tian³ Guoteng Wang³ Shenggui Li¹
Yonggang Wen¹ Zhenhua Han³ Tianwei Zhang¹

Abstract

面向 Agent 工作负载的批量推理会以持续且累积的方式对 GPU 上的键值 (KV) 缓存施加压力, 并且往往在显存容量尚未真正耗尽之前, 就已经引发严重的吞吐退化。我们将这种现象识别为**中期抖动 (middle-phase thrashing)**: 随着长生命周期 Agent 在运行过程中不断积累状态, 缓存效率在执行中段发生塌陷, 而这一病态行为此前并未被充分刻画。

我们认为, 要缓解这一问题, 系统必须从被动的请求级缓存管理转向主动的 Agent 级准入控制。受分布式系统中拥塞控制的启发, 我们将 KV 缓存视为一种共享资源, 其高效利用依赖于反馈驱动的调节机制。基于这一认识, 我们提出 Concur, 这是一个轻量级控制层, 通过调节 Agent 的准入来限制整体缓存压力, 同时保持执行连续性。Concur 使用缓存感知的控制算法, 根据运行时缓存信号动态调整活跃 Agent 数量。

在大模型与真实 Agent 工作负载上, Concur 能够避免中期抖动, 并将批量推理吞吐提升至多提升到 Qwen3-32B 上的 4.09 $\times$ 和 DeepSeek-V3 上的 1.90 $\times$, 同时保持与现有 LLM 服务系统的兼容性。

1. 引言

大语言模型 (LLM) 正越来越多地以 Agent 的形式部署, 在较长时间跨度内与外部环境交互 (Cai et al., 2025; Gao et al., 2025)。这使得高吞吐的 Agent 批量推理成为三类关键工作负载的计算基础设施: (1) Agent 强化学习 rollout, 需要大规模并行轨迹生成以支撑策略优化, 并占据端到端训练时间的主要部分 (Hu et al., 2024; Sheng et al., 2025; Chen et al., 2025d); (2) 数据蒸馏, 即从高阶推理模型蒸馏数据, 再用这些数据对学生模型进行监督微调 (Chen et al., 2025a); (3) Agent 评测, 其

中模型需要并发地模拟并评估数千种不同场景 (Zhuge et al., 2024b)。

在 Agent 批量推理中, 每个 Agent 的上下文都会随时间单调增长 (见图 1a), 从而导致其 KV 缓存占用持续扩大 (见图 1b)。随着规模提升, 驻留在 GPU 上的 KV 缓存不再只是一个静态加速结构, 而会演变为一个高度竞争、不断变化的共享资源。

现代 LLM 服务引擎通常借助前缀缓存和淘汰机制来缓解 KV 缓存压力 (Ye et al., 2024; Juravsky et al., 2024; Gim et al., 2024)。这些系统将缓存前缀组织为树结构 (Zheng et al., 2024), 并通过最近最少使用 (LRU) 策略淘汰节点, 因此在具有高前缀重叠的聊天式工作负载上表现良好。当 GPU 显存不足时, 被淘汰的 KV 缓存槽通常会被卸载到 CPU 内存, 以 PCIe 传输时延换取更少的重计算 (Qin et al., 2025; Srivatsa et al., 2024; Gim et al., 2024)。

然而, 这些技术在 Agent 批量推理场景下会失效。核心问题不仅在于上下文变长, 更在于 Agent 的推进是异步的。部分 Agent 正在主动生成 token, 而另一些 Agent 则停在工具调用阶段, 临时处于不活跃状态。采用标准 LRU 淘汰时, 这些暂时不活跃但语义上仍然关键的前缀会在内存压力升高时被激进淘汰。等 Agent 恢复执行时, 系统必须通过重计算或主机到设备传输来重建其完整前缀, 而且这一代价会随着 Agent 已积累历史的增长而增加。图 2a 在简化的三 Agent 场景中说明了这一失效模式。关键在于, 即便系统中的 Agent 总数保持不变, 这种额外开销也会被反复支付。

本文将这一病态行为识别并刻画为中期抖动 (*middle-phase thrashing*)。它不同于由简单容量耗尽导致的传统内存抖动, 而是表现为执行中一段持续较长的低效时期。基于真实部署轨迹的分析 (图 3a) 表明, Agent 批量推理具有鲜明的三阶段执行模式。系统先经历一个短暂的预热阶段, 缓存效率逐步提升; 随后进入中期阶段, 此时 GPU 显存仍处于饱和和附近, 但缓存命中率却显著塌陷。在这个阶段, 系统陷入淘汰与重计算的恶性循环, 导致吞吐显著下降。反直觉的是, 即使模型规模与硬件资源不变, 在这一阶段继续增加 Agent 数量反而会降低整体吞吐。

为应对这一挑战, 我们认为 Agent 批量推理需要从被动缓存管理转向主动的 Agent 级准入控制。我们的灵感来自分布式系统中的拥塞控制 (Jacobson, 1988; Chiu

¹ 南洋理工大学, 新加坡 ² 独立研究者 ³ 上海奇绩智峰有限公司, 中国上海. Correspondence to: Tianwei Zhang <tianwei.zhang@ntu.edu.sg>.

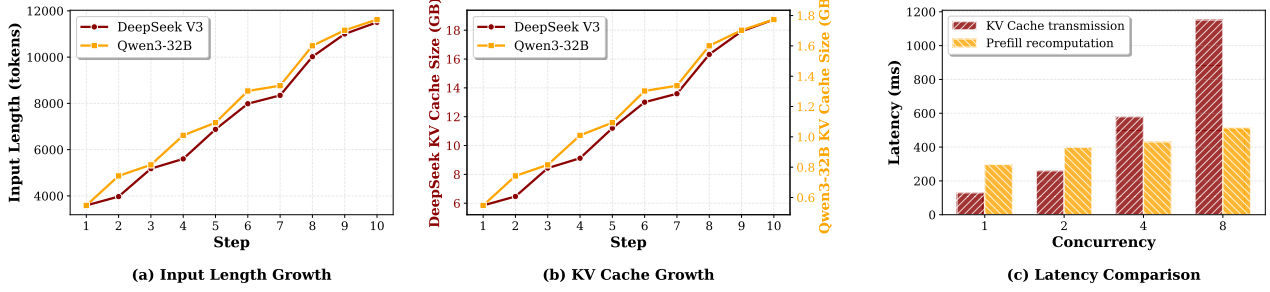


Figure 1. (a)(b) DeepSeek-V3 与 Qwen3-32B 在 10 个生成步内的输入长度与 KV 缓存显存占用增长情况。(c) 在不同并发度下, DeepSeek-V3 (每个请求 6.67 GB 缓存、4096 token) 的 GPU-CPU KV 缓存卸载时延与基于 prefill 的重计算时延对比。

& Jain, 1989; Allman et al., 2009): 访问有限共享资源时, 系统依赖反馈驱动的控制环来避免资源过载。沿着这一思路, 我们将 GPU 上的 KV 缓存视为一种共享且有限的资源, 并提出 Concur。它不是替换现有服务栈, 而是在 Agent 执行框架与现有 LLM serving engine 之间插入一个轻量级控制层, 基于缓存使用率与命中率等运行时信号动态调节活跃 Agent 的数量, 从而在维持较高利用率的同时避免中期抖动。

本文的主要贡献如下:

- 我们识别并系统刻画了 Agent 批量推理中的中期抖动现象, 说明其根源在于异步 Agent 推进、持续增长的上下文, 以及请求级被动淘汰策略之间的失配。
- 我们提出 Concur, 将拥塞控制思想引入 Agent 推理, 通过 Agent 级准入控制限制总体 KV 工作集规模, 并保持执行连续性。
- 我们在真实 Agent 工作负载与大模型上验证了该方法的有效性。Concur 在 Qwen3-32B 上可带来最高 $4.09\times$ 的吞吐提升, 在 DeepSeek-V3 上可带来最高 $1.90\times$ 的提升, 同时与现有 LLM 服务系统兼容。

2. 背景

Agent 的 ReAct 范式。大多数现代 Agent 工作负载都遵循 ReAct 范式 (Yao et al., 2022), 即 LLM 在累积上下文上持续推理, 并周期性地调用外部工具。与单轮推理不同, 这种执行方式会产生长时程、多轮交互的工作流, 过程中间思考、工具输出与观测结果会在几十乃至上百个步骤中不断追加到提示中 (Zhang et al., 2024; Zhuze et al., 2024a; Chen et al., 2025c)。

因此, 如图 1 所示, Agent 的上下文以及与之对应的 KV 缓存占用会在其生命周期内单调增长。这种无界且与步骤相关的增长, 从根本上改变了 LLM 推理的内存与执行语义, 使得 KV 缓存不再只是短生命周期的优化手段, 而成为长期存在的共享系统资源。

LLM 服务引擎中的前缀缓存。为了支持细粒度的前缀复用并消除冗余计算, 现代 LLM 服务引擎 (Zheng et al., 2024; Kwon et al., 2023) 通常将 GPU 上的 KV 缓存组织为树结构, 以尽可能复用共享前缀并降低显存占用。树中的每个节点保存一段连续 token 及其对应

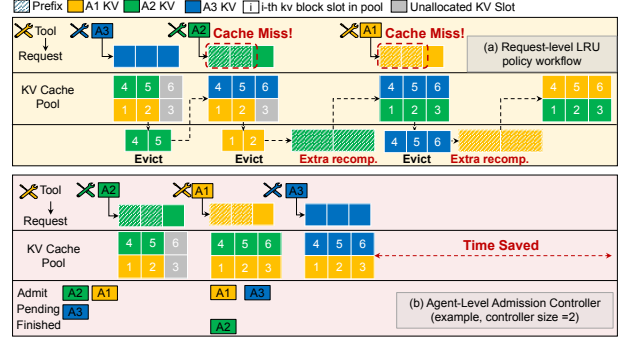


Figure 2. 三 Agent 工作流示意图, 展示中期抖动与 Agent 级准入控制。(a) LRU 淘汰使暂停中的 Agent 丢失 KV 缓存, 从而触发反复重计算并导致中期抖动。(b) Agent 级准入控制通过限制并发度避免缓存过量承诺与淘汰引发的重计算。

的 KV 缓存槽。收到新请求时, 系统从树根开始匹配前缀, 并沿匹配路径拼接 KV 缓存槽来恢复已缓存前缀。当 GPU 显存不足时, 缓存节点将通过最近最少使用 (LRU) 策略被淘汰。

这类设计对于由短、小且相互独立请求构成的工作负载很有效, 因为此类负载的前缀复用稳定且生命周期短。然而在 Agent 批量推理中, KV 缓存需求既是长期存在的, 又高度动态, 而淘汰决策仍然完全依据“最近访问时间”。更详细的分析见 §3。

为缓解 GPU 显存压力, 许多系统还会引入 CPU 内存作为二级缓存层, 将被淘汰的 KV 缓存槽通过 PCIe 卸载出去。虽然在隔离场景下, 卸载通常比重计算更快 (Jin et al., 2025; Gao et al., 2024; Qin et al., 2025; Chen et al., 2025c; 2024), 但在高并发下, 这种优势会显著下降。并发的卸载与重新加载会争抢 PCIe 带宽, 并引入额外同步开销, 导致每一步的时延像图 1c 那样迅速上升。这表明, 为低并发或请求中心型工作负载设计的 KV 缓存管理策略, 在 Agent 批量推理中很可能表现不佳。

拥塞控制与 AIMD。拥塞控制长期以来一直被视为分布式系统中调节对共享、有限资源访问的经典机制 (Jacobson, 1988)。在分组交换网络中, 多个发送方会争用有限链路带宽, 如果发送行为缺乏协调, 就会引发拥塞崩溃。为此, TCP 拥塞控制会根据能够表征拥塞

的反馈信号（如丢包或往返时延增加）动态调整发送速率。

最常见的方法之一是加性增大、乘性减小（AIMD）算法（Chiu & Jain, 1989; Floyd, 2003）。当系统未观察到拥塞时，发送方以较小的加性步长增加拥塞窗口 $cwnd$ ¹，以谨慎探测额外可用容量；一旦检测到拥塞，则通过乘性方式缩减 $cwnd$ ，从而快速释放共享资源上的压力。由于其简单且有效，AIMD 已广泛用于网络系统、数据中心调度与分布式存储等场景（Allman et al., 2009）。

本文将这种思想类比到 Agent 批量推理中的 GPU 驻留 KV 缓存容量。类似于网络链路，KV 缓存也是共享且有限的资源，一旦过量承诺，就会以效率下降、内存碎片化和执行时延上升的形式表现为“拥塞”。不同之处在于，LLM 推理中的拥塞信号更间接，也更滞后，因为资源使用是以 Agent 执行粒度演化的，而不是按 token 独立变化。正因如此，我们需要在 Agent 级别重新解释拥塞控制原则，并据此调节准入，从而在 Agent 批量推理中维持高吞吐。

3. Agent 批量推理中的中期抖动

Agent 批量推理呈现出与传统批量 LLM 服务显著不同的系统行为。本节揭示并刻画一种由 GPU 显存受限条件下 Agent 异步推进所诱发的现象，即中期抖动，并说明其为何会在高并发下导致灾难性的吞吐崩塌。我们先通过简化的工作流示例解释其根因，再用真实部署中的经验测量加以验证。

3.1. Agent 级异步性与 KV 缓存竞争

图 2a 展示了多个 Agent 共享有限 GPU 驻留 KV 缓存槽的情形。当 Agent 1 与 Agent 2（A1 和 A2）暂停去执行工具调用时，它们对应的 KV 缓存槽就暂时变为不活跃状态。按照标准 LRU 淘汰策略，这些条目会失去“最近访问”的优势，并在 Agent 3（A3）继续生成、不断占用内存时，首先成为淘汰对象。

因此，当暂停的 Agent（A1 与 A2）恢复执行时，服务引擎必须通过 prefill 重计算来重建整个前缀，这会带来显著时延。这个例子只是最简单的两方竞争情形。在存在大量 Agent 的系统中，随着各个 Agent 在主动生成和工具暂停之间反复切换，这类相互淘汰和重计算事件会更加频繁地发生。最终，级联式淘汰会在 Agent 执行的中段形成持久的缓存抖动，并演化为我们所说的中期抖动。

3.2. 通过经验分析刻画中期抖动

为了理解 Agent 批量推理的资源动态，我们分析了高并发条件下基于 DeepSeek-V3 的大规模 Agent 部署执行轨迹。图 3 给出了聚合 GPU KV 缓存使用率与对应缓存命中率的时间序列。结果表明，Agent 工作负载的

¹ $cwnd$ 是发送端的一个上限，用来控制一次可以发送多少数据，以避免压垮网络。

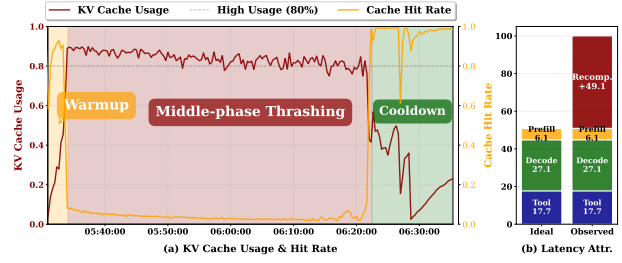


Figure 3. 真实 Agent 批量推理中的中期抖动。(a) 在 DeepSeek-V3 上运行基准测试时，端到端 KV 缓存占用随时间的演化。轨迹显示出 Agent 批量推理中典型的三阶段执行模式。(b) 同一次运行的端到端时延分解，展示 prefill、decode 与中期由 KV 缓存抖动引入的额外重计算开销所占比例。

资源消耗并非均匀分布，而是遵循一种具有代表性的三阶段模式，其主导现象正是中期抖动。

(1) 预热阶段。在初始执行阶段（图 3 中的黄色区域），Agent 仍处于较浅的推理深度，上下文较短，且提示前缀高度共享。因此，KV 缓存占用较小、相似性高，服务引擎可以最大化前缀共享，命中率接近 90%。在这个阶段，随着并发提升，吞吐通常会单调增加，因为整体工作集仍然远低于显存上限。

(2) 伴随抖动的中期阶段。随着 Agent 推进到中等长度的执行步骤，系统进入持续时间很长的中期抖动阶段，其耗时超过总执行时间的 90%。从图 3 可以看到一个关键病态：KV 缓存占用始终维持在高位（大约 80% 到 100%），但缓存命中率却快速下降并长期维持在低位。持续低命中率说明系统并不是在“有效使用”显存，而是陷入了不断淘汰与重计算的循环。我们观察到，这部分额外重计算在完整生命周期中占到端到端时延的 49.1%，与 prefill、decode 和工具调用等其他阶段相比占比极高。在这一阶段继续增加并发，反而会进一步恶化吞吐。

(3) 冷却阶段。最终，当部分 Agent 完成工作流并释放内存后，整体压力下降，淘汰频率降低，缓存命中率开始回升，系统逐渐退出抖动状态。

这些观察说明，在高吞吐 Agent 服务场景中，主要瓶颈并非峰值显存容量本身，而是现有内存管理在高度波动中期阶段仍然采取被动策略。

3.3. 对系统设计的启示

中期抖动的普遍存在揭示了当前 LLM 服务栈中的一个根本抽象失配。现有服务引擎采用请求级调度，将每一步生成都视为独立、无状态的工作单元。对于标准聊天补全任务，这种抽象是有效的；但对于 Agent 工作流，它无法表达跨多轮执行的长期状态连续性。

因此，我们认为系统设计必须沿两个维度完成范式转变：一是从请求级调度粒度转向 Agent 级调度粒度；二是从被动淘汰转向主动准入控制。这两条原则共同驱动了 Concur 的设计，下一节将详细展开。

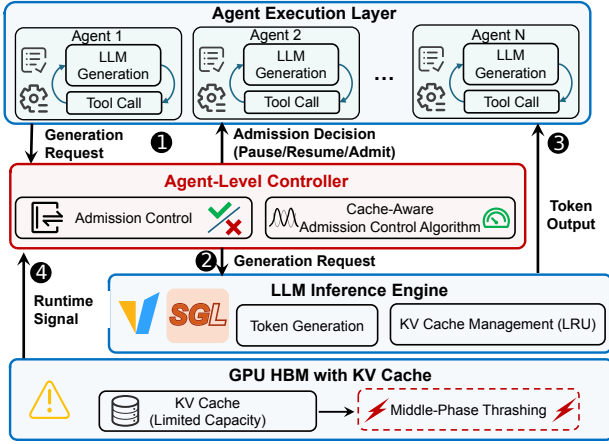


Figure 4. 系统概览。

4. Concur

4.1. 系统概览

图 4 展示了 Concur 所处的系统上下文。我们的设计并不试图重新实现一套端到端服务栈，而是在现有 Agent 执行框架与 LLM 服务引擎之间插入一个轻量级的 Agent 级控制器。整体系统可以概念性地分为三部分：Agent 执行层、Agent 级控制器和 LLM 服务引擎。

(1) **Agent 执行层**。位于系统顶层，多个 Agent 按照 ReAct 风格循环执行长时程工作流。

(2) **LLM 服务引擎**。LLM 服务引擎（例如 SGLang）负责 token 生成，并管理 GPU 显存中的 KV 缓存。

(3) **Agent 级控制器**。为了缓解中期抖动，这个控制器在 Agent 粒度上执行准入管理。与请求级调度器不同，它在多个生成步骤之间保留 Agent 的连续性。控制器持续监测实时 KV 缓存使用率与命中率，并动态调节整体活跃 Agent 并发度，以防止中期抖动。

执行流程。整个系统通过四个阶段形成闭环。① **Agent 准入**。Agent 在需要执行生成步骤时，不再直接向服务引擎发起请求，而是先提交给控制器。控制器根据当前系统状态决定准入还是暂停，并保留其执行状态。② **批量生成**。被准入的 Agent 在服务引擎中进行批量 token 生成，并按照上下文长度消耗 GPU 驻留 KV 缓存。③ **工具执行与挂起**。调用外部工具的 Agent 会暂时变为不活跃，但它们在逻辑上仍保有与自身关联的 KV 状态。控制器通过在高压力阶段限制新的准入，尽量保护这些空闲状态不被淘汰。④ **反馈与控制更新**。控制器持续采集运行时信号，尤其是 KV 缓存使用率与命中率，并据此按 §4.3 中的自适应机制更新准入策略，从而在维持吞吐的同时避免抖动。

4.2. Agent 级控制器

离线 Agent 批量推理的核心挑战在于：Agent 具有长生命周期、强状态性的执行语义，而现有 LLM 服务引擎

通常基于请求中心抽象。为弥合这一差距，我们引入一个 Agent 级控制器，在 Agent 粒度而非单次生成请求粒度上调节对服务引擎的访问。

Agent 级控制使系统能够围绕并发活跃 Agent 的整体工作集进行推理，而不只是围绕零散请求做局部决策。与其在内存压力出现后再靠缓存淘汰或重计算进行补救，控制器可以主动限制某一时刻允许执行生成步骤的 Agent 数量。这样便能让总 KV 工作集始终保持在 GPU 显存容量之内，从而直接避免中期抖动的出现。

主动准入控制。控制器实现了一套主动准入机制。当 Agent 准备执行生成步骤时，它会先把请求提交给控制器，而非直接进入服务引擎。控制器根据当前系统条件决定是否准入。被准入的 Agent 进入服务引擎并参与批量生成；未被准入的 Agent 则在 Agent 粒度上暂时暂停，同时保留其执行状态。

图 2b 给出了一个 Agent 级准入控制示例。开始时，控制器只允许 A1 和 A2 进入，使它们能够持续被推理引擎调度，同时保证二者组合后的 KV 工作集仍处于 GPU 显存预算之内。其他 Agent（如 A3）则保持等待状态，不发起推理请求，以避免 KV 缓存过量承诺。当 A2 完成执行并释放 KV 缓存后，控制器再从待执行队列中准入 A3。于是，系统始终维持至多两个活跃 Agent，从而保证内存使用稳定并避免由淘汰导致的重计算。

4.3. 缓存感知的准入控制算法

对 Agent 工作负载而言，静态准入上限在根本上是不够的。首先，Agent 的内存占用并非固定值，而是随着执行推进单调增长；一个在早期阶段安全的并发上限，迟早会在上下文拉长后触发显存饱和。其次，Agent 的推进是异步的，生成与外部工具执行交替出现，即使 Agent 总数不变，聚合内存需求也会不可预测地波动。因此，系统面临两难：过于保守的静态上限会浪费宝贵的 GPU 显存，而过于激进的上限则会跨越饱和阈值，进入抖动区间。为了在高利用率和防止抖动之间取得平衡，系统需要一个可以持续跟踪有效容量变化的动态机制。

我们将并发 Agent 的调度形式化为一个在严格内存约束下的资源分配问题，并将其与 TCP 拥塞控制进行对应。在这一类比中，共享 KV 缓存相当于网络带宽，而活跃 Agent 则是争用该有限资源的独立数据流。

问题建模。我们将 Agent 执行动态映射为如下的流量控制要素：

- (1) **拥塞窗口** (W_t) 对应于当前允许并发执行的活跃 Agent 数。
- (2) **丢包** 对应于缓存淘汰，即服务引擎因显存饱和而被迫丢弃活跃上下文。
- (3) **重传** 对应于 KV 缓存重计算。

算法设计。基于经典 TCP 中的 AIMD 算法，我们提出缓存感知的准入控制算法。它依据来自服务引擎的两个实时反馈信号调节窗口大小 W_t ：其一是 KV 缓存使

用率 (U_t), 作为前瞻性的拥塞信号; 其二是缓存命中率 (H_t), 作为反应式的失败信号。控制律定义如下:

$$W_{t+1} = \begin{cases} W_t + \alpha & \text{if } U_t < U_{low} \\ W_t \times \beta & \text{if } U_t > U_{high} \wedge H_t < H_{thresh} \\ W_t & \text{otherwise} \end{cases} \quad (1)$$

解释。该控制律中的每一项都对应 Agent 执行中的一种关键动态:

- (1) **线性探索 (α)**。当系统处于低利用状态 ($U_t < U_{low}$) 时, 采用加性增大方式线性探测未知的有效容量。这样既能提升 GPU 显存利用率, 也能避免指数式放大带来的突然过冲。
- (2) **降低二次代价 (β)**。与网络中的丢包不同, 缓存抖动引发的代价是凸性的, 因为重计算随上下文长度增长而非常昂贵。因此, 一旦检测到抖动, 控制器会执行乘性缩减, 使系统能以指数速度退出高代价区域, 从而压低累计重计算开销。
- (3) **稳定与饱和**。 U_{low} 与 U_{high} 之间的区间充当了资源分配缓冲区。由于长上下文 Agent 的准入往往会造造成离散式显存突增, 而非平滑变化, 因此该缓冲区可以避免系统因单次突增而立即跌入惩罚状态 ($U_t > U_{high}$)。同时, 只要命中率仍保持健康, 系统就可以在饱和区维持较高并发, 以优先保证吞吐。

Agent 级控制原语。为了在不与请求级调度强耦合的前提下调节 Agent 执行, 控制器暴露了三个最小化的 Agent 级原语: *admit*、*pause* 与 *resume*。它们作用于 Agent 生命周期, 而不是单次生成请求。具体而言, *admit* 允许 Agent 执行下一次生成; *pause* 在保留其执行状态的同时, 暂时阻止其继续发起请求, 并且只会在生成步骤与工具执行之间这类明确边界处生效, 以避免打断正在进行的计算; *resume* 则在资源重新可用时恢复一个已暂停 Agent 的执行, 并且由准入控制统一协调, 避免重新激化缓存拥塞。通过 *pause* 与 *resume*, 控制器能够在工作负载变化时动态调整活跃 Agent 集合。

5. 评估

为系统性评估 Concur, 我们围绕以下研究问题组织实验:

Q1: 在离线批量 Agent 推理中, Concur 可以带来多大的端到端吞吐提升? (§5.1)

Q2: 在高并发下, Concur 会如何影响 KV 缓存行为, 尤其是缓存命中率与内存利用情况? (§5.2)

Q3: 固定大小的准入控制策略是否足以支持 Agent 工作负载? (§5.3)

5.1. 端到端性能

为回答 Q1, 我们评估了不同并发水平下离线批量 Agent 推理的端到端时延。对比对象包括三类基线:

(i) **SGLang**, 采用请求级 batching 与基于 LRU 的 KV 缓存淘汰; (ii) **带请求级准入控制的 SGLang**, 通过固定请求上限限制并发; (iii) **带 HiCache 的 SGLang**, 这是一种优先保留 KV 缓存并依赖 CPU 卸载的缓存中心方案。所有系统使用相同模型权重、提示词与 rollout 工作负载。实验平台为 NVIDIA H100 GPU (80GB), 节点内通过 900 GB/s NVLink 互联, 跨节点通过 8 路 400 Gbps RoCE 互联; 软件栈包含 PyTorch 2.7.1、Python 3.12 与 CUDA 12.6。

超参数设置。在我们的实现中, 所有实验都使用统一控制参数。遵循 AIMD 拥塞控制的常见经验, 我们将加性增大因子设为 $\alpha = 2$, 乘性减小因子设为 $\beta = 0.5$, 以在稳定探测容量与快速拥塞恢复之间取得平衡。利用率与命中率阈值设置为 $U_{low} = 0.2$ 、 $U_{high} = 0.5$ 和 $H_{thresh} = 0.2$ 。附录 A.1 给出了利用率阈值的敏感性分析。结果表明, U_{high} 可以在较宽且不敏感的区间 (0.5–0.6) 内选择, 而 U_{low} 则需要更谨慎地校准, 因为它直接决定窗口增长的激进程度。我们最终采用的 $U_{low} = 0.2$ 和 $U_{high} = 0.5$ 在主动探测容量与及时响应拥塞之间取得了较好平衡。

我们在两类代表性大模型 Qwen3-32B 与 DeepSeek-V3 上报告端到端批量时延, 并通过 batch size 与张量并行 (TP) 控制有效并发。时延越低, 代表离线 Agent 推理的有效吞吐越高。表 1 总结了所有配置下的结果。

Concur 在两类模型与所有并发配置上都取得了最低时延, 并且在高并发场景下优势尤其明显。对于 Qwen3-32B, 相比原生 SGLang, Concur 的时延最高可降低 4.09 $\times$; 相比请求级准入控制, 最高可降低接近 3 $\times$ 。随着 TP 减小、每张 GPU 的有效并发提高, 基线系统会遭遇明显放缓, 而 Concur 的优势继续扩大。对于 DeepSeek-V3, Concur 在大 batch 下同样优于全部基线, 并避免了 SGLang 与缓存中心方案中常见的时延膨胀。

请求中心和缓存中心基线之所以表现不佳, 是因为它们无法让调度决策与 Agent 级内存局部性对齐。请求级准入控制看不到长期运行 Agent 所积累的 KV 工作集, 因此常常在其缓存状态已经被淘汰之后才安排其继续执行, 导致反复重计算。缓存中心方案 (如 HiCache) 虽然能通过 CPU 卸载降低重计算频率, 但在高并发条件下, PCIe 竞争与排队时延会成为主导, 从而限制其效果。

与之相对, Concur 在 Agent 粒度上调节并发, 直接限制活跃 Agent 的整体 KV 工作集规模。这种方式能在执行中期阻止过量准入, 维持较高缓存命中率, 并在整个推理过程中保持稳定的批处理效率。因此, Concur 才能在不同模型与 batch 配置上持续保持低时延。

5.2. KV 缓存行为分析

表 2 报告了与 §5.1 相同配置下, DeepSeek-V3 的平均 KV 缓存命中率。结果与此前观察到的端到端时延趋势高度一致, 并直接为中期抖动提供了证据。

Table 1. 在不断升高的有效并发度下，离线 Agent 推理的端到端时延与加速比，单位为秒。

模型	Batch / TP / #GPU	SGLang (s)	SGLang + Request Control (s)	SGLang + HiCache (s)	Concur (s)
Qwen3-32B	256 / 8 / 8	1480 (1.00×)	2049 (0.72×)	976 (1.52×)	362 (4.09×)
	256 / 4 / 4	2213 (1.00×)	1089 (2.03×)	1678 (1.32×)	757 (2.92×)
	256 / 2 / 2	2527 (1.00×)	1383 (1.83×)	1112 (2.27×)	846 (2.99×)
DeepSeek-V3	16 / 16 / 16	873 (1.00×)	861 (1.01×)	2559 (0.34×)	521 (1.68×)
	32 / 16 / 16	1226 (1.00×)	1367 (0.90×)	2277 (0.54×)	1018 (1.20×)
	40 / 16 / 16	3877 (1.00×)	2903 (1.34×)	2320 (1.67×)	2043 (1.90×)

Table 2. DeepSeek-V3 在 TP=8、8 张 GPU 条件下，不同 batch 与并发配置对应的 KV 缓存命中率 (%)。Concur 相比基线保持了更高命中率。

Batch	SGLang (%)	SGLang + HiCache (%)	SGLang + Request Control (%)	Concur (%)
16	80.38	97.48	69.00	96.38
32	77.72	97.13	68.39	93.65
40	35.41	96.08	32.21	73.36

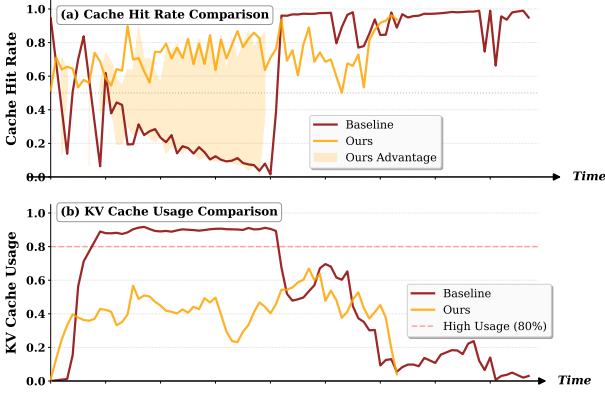


Figure 5. Qwen3-32B 在受限资源条件下 (Batch 256, TP=2, 2 张 GPU) 进行大批量离线 Agent 推理时，KV 缓存的时间动态。

随着并发提升，SGLang 的 KV 缓存命中率迅速下降。小 batch 下命中率尚可，但在 batch 为 40 的配置下会骤降到 35.41%。即使通过请求级准入限制飞行中的请求数量，请求级控制基线在高并发下的命中率仍会降到 32.21%。HiCache 通过积极保留 KV 缓存并卸载到 CPU，在所有配置下都保持了很高命中率；但如 §5.1 所示，仅有高命中率并不足以保证低时延，因为 PCIe 传输本身的额外开销会抵消这一优势。

相比之下，Concur 在避免过度卸载的同时仍保持了较高 KV 缓存命中率。由于它在 Agent 粒度上控制并发，能够在抖动阶段保留更好的 KV 局部性，同时避免显存过量承诺。这种平衡正是其在 Q1 中实现最低时延并显著优于 SGLang 与请求级基线的原因。

为了更细致地理解时间动态，图 5 给出了 batch size 为 256 时 Agent 推理全过程中的 KV 缓存命中率 (上) 与

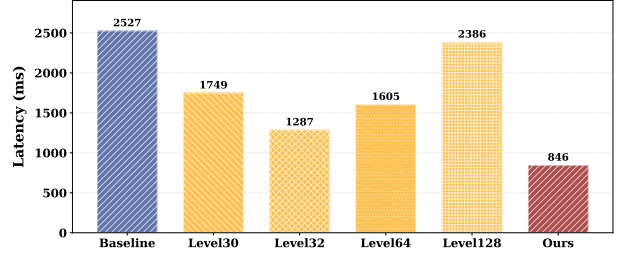


Figure 6. Qwen3-32B (Batch 256, TP 2, 2 张 GPU) 在固定准入控制与自适应准入控制下的端到端时延对比。

使用率 (下)。当多个 Agent 同步推进到中等长度步骤时，KV 缓存使用率接近容量上限 (约 80%)，而基线方法的命中率会急剧下降。Concur 通过调节 Agent 准入维持明显更高的命中率，从而避免多个 Agent 同时对 KV 缓存槽造成过量承诺。

5.3. 静态准入控制与缓存感知准入控制

为回答 Q3，我们评估固定大小的准入控制是否足以支撑 Agent 工作负载。实验中我们测试了多个固定准入水平 (30、32、64、128)，并与自适应策略 Concur 对比。

图 6 给出了结果。较小的固定水平 (如 30–64) 相比完全不控的基线能够降低时延，但往往过于保守，会在某些执行阶段低估可用 GPU 资源。较大的固定水平 (如 128) 则允许更多并发，却会导致显存过量承诺，引发 KV 缓存抖动并推高时延。Concur 取得了最低观测时延 846 ms，相比最佳固定水平配置可进一步提升约 1.5–2.9×，相对无控制基线提升 2.99×。

总的来说，静态准入控制是一种脆弱策略：要么浪费 GPU 资源，要么触发 KV 缓存抖动，其表现高度依赖具体阈值选择。相比之下，Concur 通过阶段感知的并发调节，在 Agent 工作负载中同时实现了高吞吐与稳定执行，而这正是固定大小控制难以做到的。

6. 相关工作

解耦内存与前缀缓存。已有工作尝试将 KV 缓存管理从计算中解耦，以缓解碎片化和负载不均问题。例如 TokenLake (Wu et al., 2025) 提出了带统一前缀池的解耦式内存架构，用于减少不同实例之间的冗余。尽管这

类方法能够改善空间利用效率，但它们并未处理离线 Agent 推理中固有的时间维度上的过量承诺问题。在该场景下，并发 Agent 会在中期同时扩展上下文，即使采用统一缓存池，也仍可能被压垮。我们的工作与其互补，通过引入主动流量控制来避免内存饱和。

面向应用结构的 Serving。近期一些系统开始超越请求级调度，通过利用应用结构来降低时延和任务完成时间。Parrot (Lin et al., 2024)、Teola (Tan et al., 2024)、Autellix (Luo et al., 2025) 与 Kairos (Chen et al., 2025b) 等方法利用 DAG、依赖感知或内存预测来重排请求，以减轻多轮工作负载中的阻塞。它们通常假设内存压力可以通过淘汰或重排来处理。相比之下，我们关注的是大批量离线场景，在这里无论调度顺序如何，过高并发都会引发抖动，因此系统需要显式的准入控制。

Agent 原生推理与并发控制。Agent 执行在 GPU 生成与 CPU 侧工具调用之间交替进行，这使内存管理更为复杂。TokenCake (Bian et al., 2025) 和 Continuum (Li et al., 2025) 主要依赖卸载、基于 TTL 的缓存固定等被动机制来处理这一问题。我们的方案则有本质差异：受网络拥塞控制启发，我们使用 AIMD 主动调节 Agent 并发，以限制内存压力并消除中期抖动，而不是依赖代价高昂的交换操作。

7. 结论

Agent 批量推理会持续、累积地对 GPU KV 缓存施加压力，从而暴露出现有请求级调度与被动内存管理难以处理的性能病态。本文识别出离线 Agent 工作负载中的一个关键瓶颈，即中期抖动，并据此提出需要转向主动的 Agent 级准入控制。受拥塞控制启发，我们设计了 Concur，它是一个轻量级系统层，通过缓存反馈调节 Agent 并发，以稳定内存效率并提升吞吐。实验结果表明，在 Agent 粒度上控制并发既有效也实用，这也表明，受流量控制启发的机制在可扩展 LLM 推理系统中具有更广泛的潜力。

影响声明

本文旨在推动机器学习系统的发展。我们认为本工作的潜在社会影响与一般的大规模模型基础设施研究相似，没有需要在此额外单独强调的特殊影响。

References

- Allman, M., Paxson, V., and Blanton, E. Tcp congestion control. Technical report, 2009.
- Bian, Z., Wu, F., Ma, T., and Zhuo, Y. Tokencake: A kv-cache-centric serving framework for llm-based multi-agent applications. *arXiv preprint arXiv:2510.18586*, 2025.
- Cai, Y., Chen, L., Chen, Q., Ding, Y., Fan, L., Fu, W., Gao, Y., Guo, H., Guo, P., Han, Z., et al. Nex-n1: Agentic models trained via a unified ecosystem for large-scale environment construction. *arXiv preprint arXiv:2512.04987*, 2025.
- Chen, H., Tu, H., Wang, F., Liu, H., Tang, X., Du, X., Zhou, Y., and Xie, C. Sft or rl? an early investigation into training rl-like reasoning large vision-language models, 2025a. URL <https://arxiv.org/abs/2504.11468>.
- Chen, J., Shi, J., Chen, Q., and Guo, M. Kairos: Low-latency multi-agent serving with shared llms and excessive loads in the public cloud. *arXiv preprint arXiv:2508.06948*, 2025b.
- Chen, Q., Gu, D., Wang, G., Chen, X., Xiong, Y., Huang, T., Hu, Q., Jin, X., Wen, Y., Zhang, T., et al. Internevo: Efficient long-sequence large language model training via hybrid parallelism and redundant sharding. *arXiv preprint arXiv:2401.09149*, 2024.
- Chen, Q., Li, S., Gao, W., Sun, P., Wen, Y., and Zhang, T. Sppo: Efficient long-sequence llm training via adaptive sequence pipeline parallel offloading. *arXiv preprint arXiv:2503.10377*, 2025c.
- Chen, Q., Liu, Z., Sun, P., Li, S., Wang, G., Liu, Z., Wen, Y., Feng, S., and Zhang, T. Respec: Towards optimizing speculative decoding in reinforcement learning systems. *arXiv preprint arXiv:2510.26475*, 2025d.
- Chiu, D.-M. and Jain, R. Analysis of the increase and decrease algorithms for congestion avoidance in computer networks. *Computer Networks and ISDN systems*, 17(1): 1–14, 1989.
- Floyd, S. Highspeed tcp for large congestion windows. Technical report, 2003.
- Gao, B., He, Z., Sharma, P., Kang, Q., Jevdjic, D., Deng, J., Yang, X., Yu, Z., and Zuo, P. {Cost-Efficient} large language model serving for multi-turn conversations with {CachedAttention}. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, pp. 111–126, 2024.
- Gao, W., Zhao, Y., An, D., Wu, T., Cao, L., Xiong, S., Huang, J., Wang, W., Yang, S., Su, W., et al. Rollpacker: Mitigating long-tail rollouts for fast, synchronous rl post-training. *arXiv preprint arXiv:2509.21009*, 2025.
- Gim, I., Chen, G., Lee, S.-s., Sarda, N., Khandelwal, A., and Zhong, L. Prompt cache: Modular attention reuse for low-latency inference. *Proceedings of Machine Learning and Systems*, 6:325–338, 2024.
- Hu, J., Wu, X., Shen, W., Liu, J. K., Zhu, Z., Wang, W., Jiang, S., Wang, H., Chen, H., Chen, B., et al. Open-rlhf: An easy-to-use, scalable and high-performance rlhf framework. *arXiv preprint arXiv:2405.11143*, 2024.

- Jacobson, V. Congestion avoidance and control. In *Symposium Proceedings on Communications Architectures and Protocols*, SIGCOMM '88, pp. 314–329, New York, NY, USA, 1988. Association for Computing Machinery. ISBN 0897912799. doi: 10.1145/52324.52356. URL <https://doi.org/10.1145/52324.52356>.
- Jin, C., Zhang, Z., Jiang, X., Liu, F., Liu, S., Liu, X., and Jin, X. Ragcache: Efficient knowledge caching for retrieval-augmented generation. *ACM Transactions on Computer Systems*, 44(1):1–27, 2025.
- Juravsky, J., Brown, B., Ehrlich, R., Fu, D. Y., Ré, C., and Mirhoseini, A. Hydragen: High-throughput llm inference with shared prefixes. *arXiv preprint arXiv:2402.05099*, 2024.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pp. 611–626, 2023.
- Li, H., Mang, Q., He, R., Zhang, Q., Mao, H., Chen, X., Cheung, A., Gonzalez, J., and Stoica, I. Continuum: Efficient and robust multi-turn llm agent scheduling with kv cache time-to-live. *arXiv preprint arXiv:2511.02230*, 2025.
- Lin, C., Han, Z., Zhang, C., Yang, Y., Yang, F., Chen, C., and Qiu, L. Parrot: Efficient serving of {LLM-based} applications with semantic variable. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pp. 929–945, 2024.
- Luo, M., Shi, X., Cai, C., Zhang, T., Wong, J., Wang, Y., Wang, C., Huang, Y., Chen, Z., Gonzalez, J. E., et al. Autellix: An efficient serving engine for llm agents as general programs. *arXiv preprint arXiv:2502.13965*, 2025.
- Qin, R., Li, Z., He, W., Cui, J., Ren, F., Zhang, M., Wu, Y., Zheng, W., and Xu, X. Mooncake: Trading more storage for less computation—a {KVCache-centric} architecture for serving {LLM} chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, pp. 155–170, 2025.
- Sheng, G., Tong, Y., Wan, B., Zhang, W., Jia, C., Wu, X., Wu, Y., Li, X., Zhang, C., Peng, Y., Lin, H., Liu, X., and Wu, C. Laminar: A scalable asynchronous rl post-training framework, 2025. URL <https://arxiv.org/abs/2510.12633>.
- Srivatsa, V., He, Z., Abhyankar, R., Li, D., and Zhang, Y. Preble: Efficient distributed prompt scheduling for llm serving. *arXiv preprint arXiv:2407.00023*, 2024.
- Tan, X., Jiang, Y., Yang, Y., and Xu, H. Teola: Towards end-to-end optimization of llm-based applications. *arXiv preprint arXiv:2407.00326*, 2024.
- Wu, B., Zhang, Z., Zhong, Y., Huang, G., Zhu, Y., Liu, X., and Jin, X. Tokenlake: A unified segment-level prefix cache pool for fine-grained elastic long-context llm serving. *arXiv preprint arXiv:2508.17219*, 2025.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. R., and Cao, Y. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.
- Ye, L., Tao, Z., Huang, Y., and Li, Y. Chunkattention: Efficient self-attention with prefix-aware kv cache and two-phase partition. *arXiv preprint arXiv:2402.15220*, 2024.
- Zhang, J., Xiang, J., Yu, Z., Teng, F., Chen, X., Chen, J., Zhuge, M., Cheng, X., Hong, S., Wang, J., et al. Aflow: Automating agentic workflow generation. *arXiv preprint arXiv:2410.10762*, 2024.
- Zheng, L., Yin, L., Xie, Z., Sun, C. L., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., et al. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems*, 37:62557–62583, 2024.
- Zhuge, M., Wang, W., Kirsch, L., Faccio, F., Khizbullin, D., and Schmidhuber, J. Gptswarm: Language agents as optimizable graphs. In *Forty-first International Conference on Machine Learning*, 2024a.
- Zhuge, M., Zhao, C., Ashley, D., Wang, W., Khizbullin, D., Xiong, Y., Liu, Z., Chang, E., Krishnamoorthi, R., Tian, Y., Shi, Y., Chandra, V., and Schmidhuber, J. Agent-as-a-judge: Evaluate agents with agents, 2024b. URL <https://arxiv.org/abs/2410.10934>.

Table 3. Qwen3-32B 在不同张量并行配置下，利用率阈值对推理时延（ms）的敏感性分析。最优配置 $(U_{\text{low}}, U_{\text{high}}) = (0.2, 0.5)$ 已用粗体标出。

变化 U_{high} ($U_{\text{low}} = 0.2$)					变化 U_{low} ($U_{\text{high}} = 0.5$)				
U_{low}	U_{high}	TP8	TP4	TP2	U_{low}	U_{high}	TP8	TP4	TP2
0.2	0.4	529	1089	1390	0.1	0.5	2894	2561	972
0.2	0.5	362	757	846	0.2	0.5	362	757	846
0.2	0.6	386	775	945	0.3	0.5	779	1566	1008
0.2	0.8	1898	2300	2498	0.5	0.5	2763	2245	1451

A. 附录

A.1. 利用率阈值敏感性分析

我们进行了系统性的敏感性分析，以评估 KV 缓存使用率阈值对端到端时延的影响。如表 3 所示，我们考察了两个维度：(1) 固定 $U_{\text{low}} = 0.2$ ，改变上界阈值 U_{high} ；(2) 固定 $U_{\text{high}} = 0.5$ ，改变下界阈值 U_{low} 。性能在 Qwen3-32B 的不同张量并行（TP）配置上测量。

U_{high} 的影响。在固定 $U_{\text{low}} = 0.2$ 时，我们观察到适中的 U_{high} （0.5–0.6）能够在所有 TP 配置下稳定地获得较低时延，且相互之间差异很小。例如，将 U_{high} 从 0.5 调整到 0.6，只会带来 24 ms（TP8）、18 ms（TP4）和 99 ms（TP2）的时延增加，说明这一范围具有较好鲁棒性。相反，过于激进的阈值（如 $U_{\text{high}} = 0.8$ ）会显著恶化性能，使时延增加 4–5 $\times$ ，其原因在于控制器在触发窗口缩减前容忍了过高缓存使用率，导致过量准入并引发缓存抖动与重计算开销。另一方面，过于保守的阈值（如 $U_{\text{high}} = 0.4$ ）又会过早响应缓存压力，提前限制准入，最终因缓存容量利用不足而带来 46–64% 的额外时延。

U_{low} 的影响。在固定 $U_{\text{high}} = 0.5$ 时，系统对 U_{low} 更敏感。如果 U_{low} 过低（0.1），缓存使用率很难真正跌破这一阈值，控制器无法提升窗口，系统因此长期停留在低并发区间，使 TP8 与 TP4 配置的时延高出 7–8 $\times$ 。相反，如果 U_{low} 设得过高（0.3 或 0.5），控制器会在缓存使用率已不低时继续增长窗口，导致过量准入，并因缓存压力过大使时延增加 2–3 $\times$ 。最优配置 $(U_{\text{low}}, U_{\text{high}}) = (0.2, 0.5)$ 则在“当缓存确实空闲时积极探测容量”和“当使用率已处于健康水平时避免过早增长”之间取得了恰当平衡。

这些结果表明， U_{high} 可以从较宽的工作区间（0.5–0.6）中选择，而 U_{low} 则需要更仔细地校准，才能避免控制器在容量利用不足与过量准入之间摇摆。